

Activity 1: Github/Tests

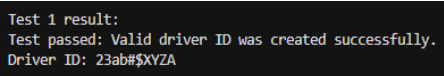
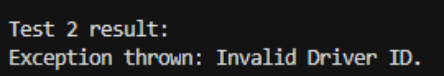
The github repository containing code and unit tests can be found online [here](#).

Driver Unit Tests

Note: print statements were used as output messages to clearly outline the result of each test.

Example:

```
@Test // test 2: an invalid Driver ID is not created
void shouldThrowErrorInvalidDriverID() {
    Exception error = assertThrows(InvalidArgumentException.class, () -> {
        new Driver (
            driverID: "invalidID",
            name: "Jane Doe",
            experienceYears: 3,
            licenseType: "Light",
            address: "456 | Elm Street | Melbourne | VIC | Australia",
            birthDate: "02-02-1990",
            age: 55
        );
    });
    System.out.println(x: "Test 2 result:");
    System.out.println("Exception thrown: " + error.getMessage());
    System.out.println();
}
```

Test Cases	Test Data	Expected Result	Test Result	Pass/Fail
D1: Normal valid driver ID	DriverID: 23ab#\$XYZA	Terminal should output: Test passed: Valid driverID was created successfully Driver ID: 23ab#\$XYZA		PASS
D1: Invalid driver ID should fail	driverID: invalidID	Terminal should output: Exception thrown: Invalid driver ID		PASS

D1: Edge case driver ID should pass	driverID: 24ab!@CDXY	Terminal should output: Test passed: Driver with valid ID was created successfully Driver ID: 24ab!@CDXY	Test 3 result: Test passed: Valid driver ID was created successfully. Driver ID: 24ab!@CDXY	PASS
D2: Valid address should pass	Address: 321 Pine Street Melbourne VIC Australia	Terminal should output: "Test passed: Address with correct format was created successfully Address: 321 Pine Street Melbourne VIC Australia	Test 4 result: Test passed: Valid address was accepted. Address: 321 Pine Street Melbourne VIC Australia	PASS
D2: Invalid address should fail	Address: Invalid Address	Terminal should output: Exception thrown: Invalid address format	Test 5 result: Exception thrown: Invalid address format.	PASS
D2: Edge case: Very short address should pass	Address: 1 Main St Melbourne VIC Australia	Terminal should output: Test passed: Address with correct format was created successfully. Address: 1 Main St Melbourne VIC Australia	Test 6 result: Test passed: Valid address was created successfully. Address: 1 Main St Melbourne VIC Australia	PASS
D3: Valid birthdate should pass	birthDate: 07-07-1955	Terminal should output: Test passed: Valid birth date was accepted. Birth Date: 07-07-1955	Test 7 result: Test passed: Valid birth date was accepted. Birth Date: 07-07-1955	PASS
D3: Invalid birthdate should fail	birthDate: invalidDate	Terminal should output: Exception thrown: Invalid birth date format.	Test 8 result: Exception thrown: Invalid birthdate format.	PASS
D3: Edge case should pass (very old birthdate)	birthDate: 01-01-1900	Terminal should output: Test passed: Valid birthdate was accepted.	Test 9 result: Test passed: Valid birth date was accepted. Birth Date: 01-01-1900	PASS

		Birth Date: 01-01-1900		
D4: Updating a non-immutable field such as address should pass	driverID: 38wx#\$YZTU Name: Noah Taylor experienceYears: 5 licenseType: Heavy Address: 123 Old Street Melbourne VIC Australia birthDate: 01-01-1980 Age: 55	Terminal should output: Test passed: Driver address was updated successfully. New Address: 456 New Street Melbourne VIC Australia	Test 13 result: Test passed: Driver address was updated successfully. New Address: 456 New Street Melbourne VIC Australia	PASS
D4: Changing an immutable field like name should fail	driverID: 49mn!@PQRS Name: John Doe experienceYears: 5 licenseType: Heavy Address: 123 Main Street Melbourne VIC Australia birthDate: 01-01-1980 Age: 55	Terminal should output: Test failed: Driver name was not updated. Driver Name: John Doe	Test 14 result: Test failed: Driver name was not updated. Driver Name: John Doe	PASS
D4: Changing an immutable field like driverID should fail	driverID: 45yz#\$ABCD Name: John Doe experienceYears: 5 licenseType: Heavy Address: 123 Main Street Melbourne VIC Australia birthDate: 01-01-1980 Age: 55	Terminal should output: Test failed: Driver ID was not updated. Driver ID: 45yz#\$ABCD	Test 15 result: Test failed: Driver ID was not updated. Driver ID: 45yz#\$ABCD	PASS
D5: Driver with 10 years or less of experience can change license	experienceYears: 7	Terminal should output: Test passed: License was changed successfully. New License Type: Heavy	Test 10 result: Test passed: License was changed successfully. New License Type: Heavy	PASS
D5: Driver with more than 10 years of experience cannot change	experienceYears: 11	Terminal should output: Exception thrown: License	Bigger image underneath table	PASS

license		type cannot be updated for drivers with more than 10 years of experience.	Test 11 result: Exception thrown: License type cannot be updated for drivers with more than 10 years of experience.	
D5: Driver with exactly 10 years of experience should allow license (edge)	experienceYears: 10	Terminal should output: Test passed: License was changed successfully. New License Type: Medium	Test 12 result: Test passed: License was changed successfully. New License Type: Medium	PASS

Test 11 result:
Exception thrown: License type cannot be updated for drivers with more than 10 years of experience.

Bus Unit Tests

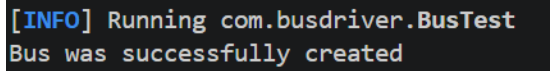
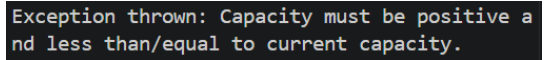
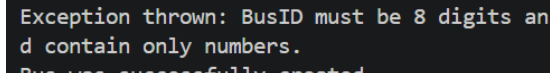
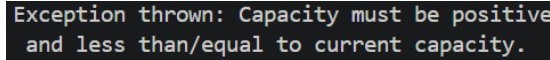
The rows in this test table appear in the BusTest.java file in the exact same order (eg: row 1 of this table corresponds to the first test written in BusTest.java).

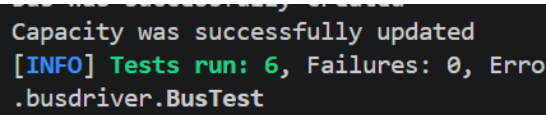
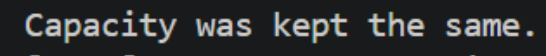
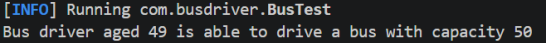
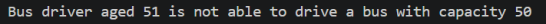
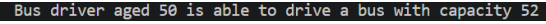
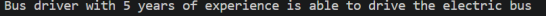
Please note that print statements were written in the tests so that output of the tests could be clearly seen. These print statements are what the expected result and the test result values are based on. An example can be seen below:

```
@Test //test 2 description: a new Bus object should be created
void shouldCreateBus() {

    Bus result = assertDoesNotThrow(() -> {
        return new Bus(busID: "12345678", capacity: 20, fuelLevel: 100);
    });

    if (result != null) {
        System.out.println(x: "Bus was successfully created");
    }
    else {
        System.out.println(x: "Bus was not created");
    }
}
```

Test Cases	Test Data	Expected Result	Test Result	Pass/Fail
Constructing Bus: Invalid bus ID and invalid parameters	busID = '0' capacity = -1 fuelLevel = -1.0 fuelType = 'incorrect'	Terminal should have output "Exception thrown: BusID must be 8 digits, and contain only numbers" after 'mvn test' is run	The text 'Exception thrown: BusID must be 8 digits, and contain only numbers' was shown in the terminal.	PASS
Constructing Bus: Valid bus ID and valid parameters	busID = '12345678' capacity = 20 fuelLevel = 20.0 fuelType = 'diesel'	Terminal should have output "Bus was successfully created" after 'mvn test' is run	The text 'bus was successfully created' was shown in terminal. 	PASS
Constructing Bus: Valid bus ID and invalid parameters	busID = '88888888' capacity = -100 fuelLevel = -100.10 fuelType = 'diesel'	Terminal should have output "Exception thrown: Capacity must be positive and less than/equal to current capacity." after 'mvn test' is run	The text 'Exception thrown: Capacity must be positive and less than/equal to current capacity' was seen in terminal. 	PASS
Constructing Bus: Invalid bus ID data type and valid parameters	busID = 'incorrectID' capacity = 200 fuelLevel = 155.55 fuelType = 'hybrid'	Terminal should have output "Exception thrown: BusID must be 8 digits, and contain only numbers" after 'mvn test' is run	The text 'Exception thrown: BusID must be 8 digits, and contain only numbers' appeared in terminal after 'mvn test' command was entered. 	PASS
Updating capacity: new <i>increased</i> capacity	oldCapacity = 75 newCapacity = 76	Terminal should have output "Exception thrown: Capacity must be positive and less than/equal to current capacity." after 'mvn test' is run	The text 'Exception thrown: Capacity must be positive and less than/equal to current capacity' was present in terminal. 	PASS
Updating capacity: new <i>decreased</i> capacity	oldCapacity = 100 newCapacity = 99	Terminal should have output "Capacity was successfully updated" after 'mvn test' is run	The text 'Capacity was successfully updated' appeared in the terminal after the 'mvn test' command was entered.'	PASS

				
Updating capacity: capacity stays the same	oldCapacity = 200 newCapacity = 200	Terminal should have output "Capacity was kept the same." after 'mvn test' is run	The text 'Capacity was kept the same.' appeared in the terminal after the 'mvn test' command was entered. 	PASS
Driver age restriction: driver is less than 50 years old and bus capacity is 50 or more	age = 49 capacity = 50	Terminal should have output "Bus driver aged 49 is able to drive a bus with capacity 50 " after 'mvn test' is run	The text 'Bus driver aged 49 is able to drive a bus with capacity 50' appeared in the terminal after the 'mvn test' command was entered. 	PASS
Driver age restriction: driver is more than 50 years old and bus capacity is 50 or more	age = 51 capacity = 51	Terminal should have output "Bus driver aged 51 is not able to drive a bus with capacity 51" after 'mvn test' is run	The text 'Bus driver aged 51 is able to drive a bus with capacity 51' appeared in the terminal after the 'mvn test' command was entered. 	PASS
Driver age restriction: driver is 50 years old and bus capacity is 50 or more	age = 50 capacity = 52	Terminal should have output "Bus driver aged 50 is able to drive a bus with capacity 52" after 'mvn test' is run	The text 'Bus driver aged 50 is able to drive a bus with capacity 52' appeared in the terminal after the 'mvn test' command was entered. 	PASS
Electric bus restriction: driver has 5 years of experience and bus fuelType is electric	experienceYears = 5 fuelType = 'electricity'	Terminal should have output "Bus driver with 5 years of experience is able to drive the electric bus" after 'mvn test' is run	The text 'Bus driver with 5 years of experience is able to drive the electric bus' appeared in the terminal after the 'mvn test' command was entered. 	PASS
Electric bus restriction:	experienceYears = 6	Terminal should have output "Bus	The text 'Bus driver with 6 years of	PASS

driver has more than 5 years of experience and bus fuelType is electric	fuelType = 'electricity'	driver with 6 years of experience is able to drive the electric bus" after 'mvn test' is run	experience is able to drive the electric bus' appeared in the terminal after the 'mvn test' command was entered. Bus driver with 6 years of experience is able to drive the electric bus	
Electric bus restriction: driver has less than 5 years of experience and bus fuelType is electric	experienceYears = 4 fuelType = 'electricity'	Terminal should have output "Bus driver with 4 years of experience is not able to drive the electric bus" after 'mvn test' is run	The text 'Bus driver with 4 years of experience is not able to drive the electric bus' appeared in the terminal after the 'mvn test' command was entered. Bus driver with 4 years of experience is not able to drive the electric bus	PASS
Driver license restriction: driver has Heavy license and bus fuelType is electric	licenseType = 'Heavy' fuelType = 'electricity'	Terminal should have output "Bus driver with Heavy license is able to drive electric bus" after 'mvn test' is run	The text 'Bus driver with Heavy license is able to drive electric bus' appeared in the terminal after the 'mvn test' command was entered. Bus driver with Heavy license is able to drive electric bus	PASS
Driver license restriction: driver has Public Transport license and bus fuelType is hybrid	licenseType = 'PublicTransport' fuelType = 'hybrid'	Terminal should have output "Bus driver with Public Transport license is able to drive hybrid bus" after 'mvn test' is run	The text 'Bus driver with Public Transport license is able to drive hybrid bus' appeared in the terminal after the 'mvn test' command was entered. Bus driver with Public Transport license is able to drive hybrid bus	PASS
Driver license restriction: driver has Light license and bus fuelType is electric	licenseType = 'Light' fuelType = 'electricity'	Terminal should have output "Bus driver with Light license is not able to drive electric bus" after 'mvn test' is run	The text 'Bus driver with Light license is not able to drive electric bus' appeared in the terminal after the 'mvn test' command was entered. Bus driver with Light license is not able to drive electric bus	PASS
Driver license restriction: driver has medium license and bus fuelType is hybrid	licenseType = 'Medium' fuelType = 'hybrid'	Terminal should have output "Bus driver with Medium license is not able to drive hybrid bus" after 'mvn test' is run	The text 'Bus driver with Medium license is not able to drive hybrid bus' appeared in the terminal after the 'mvn test' command was entered. Bus driver with Medium license is not able to drive hybrid bus	PASS

Driver Repository Integration Tests

Test Cases	Test Data	Expected Result	Actual Result	Pass/Fail
Driver repository validation: Add a driver, reload from file and verify all details.	driverID = 23ab#\$XYZA, name = "John Doe", experienceYears = 5, licenseType = "Heavy", address = "123 Main Street Melbourne VIC Australia", birthDate = "01-01-1980", age = 55	Returns the same details that have been added	Driver is created and retrievable. "Running Test 1, Adding valid driver with ID: 23ab#\$XYZA, name: John Doe, experience: 5, license: Heavy, Retrieved data: 23ab#\$XYZA, John Doe, 5, Heavy, 123 Main Street Melbourne VIC Australia, 01-01-1980, Test 1 Passed."	PASS
Driver validation: Add drivers with an invalid ID and a duplicate ID, verify both are rejected	Driver 1 (invalid): driverID = "invalidID", name = "Jane Doe", experience = 3, licenseType = "Light", address = "456 Elm Street Melbourne VIC Australia", birthDate = "02-02-1990", age = 55. Driver 2 (duplicate): driverID = "24ab!@CDXY" added twice	Not added to the repository due to prior validation in classes	Invalid and duplicate drivers are rejected. "Rejected driver with invalid ID: Invalid Driver ID. Rejected driver with duplicate ID: Driver ID already exists. Test 2 passed. Repo count: 1"	PASS
Driver update: Add a driver, update their address and experience, reload from file and verify changes are persisted	driverID = "35cd!@XCSA", name = "Judy Rock", experience = 9, licenseType = "Light", address = "34 La Trobe Street Melbourne VIC Australia", birthDate = "01-01-1990", age = 34. Updated address = "99 Exhibition Street Melbourne VIC Australia", updated experience = 8, updated	Details of the driver are updated to new values whilst retaining the same ID	Driver is updated with new values. "Running test 3, Address before update: 34 La Trobe Street Melbourne VIC Australia, Experience before update: 9, Address after update: 99 Exhibition Street Melbourne VIC Australia, Experience after	PASS

	license = "Heavy"		update: 8, Test 3 passed"	
Driver count: Add two drivers, check the count, add two more, reload from file and verify final count is 4	Driver 1: driverID = "24ab!@XCSA", name = "Judy Rock", experience = 9, licenseType = "Light", age = 34. Driver 2: driverID = "23ab@#CDBA", name = "Sam Jones", experience = 10, licenseType = "Heavy", age = 44. Driver 3: driverID = "56er\$%DFRE", name = "Bob Jones", experience = 12, licenseType = "Light", age = 39. Driver 4: driverID = "67yu#\$SEDR", name = "Sally Brown", experience = 7, licenseType = "Heavy", age = 44	The driver counts are updated and equal the amount added to the repository	Driver count is updated. "Running test 4, Added two drivers, repo count: 2, Added two more drivers, repo count: 4, Test 4 passed. Final repo count: 4"	PASS

Bus Repository Integration Tests

Test Cases	Test Data	Expected Result	Actual Result	Pass/Fail
Bus repository validation: Add a new bus. Check the bus in the repository with the ID.	busID = 87654321 capacity = 50 fuelLevel = 20.0 fuelType = "diesel"	Returns the same details that has been added	The bus is created and is retrievable. "Running Test 1 Adding a valid bus with ID: 87654321, capacity 50, fuelLevel: 20.0, fuelType: diesel Retrieving bus with ID: 87654321"	PASS

Bus validation: Add a bus with incorrect details	busID = 8765432 capacity = -50 fuelLevel = -20.0 fuelType = "fiesel"	Not added to the repository due to prior validation in classes	Bus is thrown out "Running Test 2 Creating an invalid bus with ID: 8765432, capacity: -50, fuelLevel: -20.0, fuelType: fiesel Test 2 Passed: Bus was throw out"	PASS
Bus update: Retrieve the details of a bus, update it and review the details.	busID = 04052016 Capacity = 52 fuelLevel = 25 fuelType = electric Updated: Capacity = 40 fuelLevel = 15 fuelType = hybrid	The details of the bus are updated to their new values whilst retaining the same id.	Bus is updated with new values. "Test 3 running: Bus ID: 04052016, capacity: 52, fuelLevel: 25.0, fuelType: diesel Updating capacity to 40 Updating fuel level to 15 Updating fuel type to hybrid UPDATED DETAILS: Bus ID: 04052016, capacity: 40, fuelLevel: 15.0, fuelType: hybrid Test 3 passed"	PASS
Bus count: Get the record count, and then add two more buses to the repository. Retrieve the count again.	Bus 1: busID = 04121999 Capacity = 22 fuelLevel = 11 fuelType = diesel Bus 2: busID = 19962608 Capacity = 44	The bus counts are updated and equal the amount added to the repository.	Bus count is updated: "Test 4 running Added two initial buses repo count: 2 Added two more buses repo count: 4"	PASS

	fuelLevel = 15 fuelType = hybrid Bus 3: busID = 20030901 Capacity = 12 fuelLevel = 5 fuelType = electricity Bus 4: busID = 01121997 Capacity = 100 fuelLevel = 100 fuelType = hybrid			
--	---	--	--	--

Activity 2: User Stories and Acceptance Criteria

User types in the user stories:

- **Operations Administrator**: This user type refers to users who oversee the bus network's operations. As part of their role, they can perform CRUD operations for buses and drivers, altering the bus and driver network as needed.
- **Bus Driver**: This user types operates a bus in the bus network.
- **Maintenance Technician**: This user evaluates the condition of buses, and updates the status of details of them.
- **Bus Dispatcher**: This user assigns buses to different routes in the network and updates them depending on situations.

User Story 1: As an operations administrator, I want to add a new bus into the system, so that more buses will be able to provide transport services in the transport network.

- **Acceptance Criteria 1**: Given a bus has an id that is exactly 8 characters long and only contains digits, when the administrator attempts to add the bus into the system, then the system should add the bus into the bus repository.
- **Acceptance Criteria 2**: Given a bus has an id that is duplicate to an existing bus in the bus repository, when the administrator attempts to add the bus into the system, then the system should reject the addition of the bus, and not add it into the bus repository.

- **Acceptance Criteria 3:** Given a bus with positive capacity and fuel level, as well as a fuel type that is either 'diesel', hybrid' or 'electricity', when the administrator attempts to add the bus into the system, then the system should add the new bus into the repository.

User Story 2: As a bus driver, I want to check my eligibility to drive buses (such as my age, license type, and bus type), so that I can safely and legally drive the buses best suited for me.

- **Acceptance Criteria 1:** Given a driver is 50 years old or younger, when the driver checks their eligibility to drive buses with a capacity of 50 or more, the system should mark the driver eligible and permit them to operate such buses.
- **Acceptance Criteria 2:** Given a driver has at least 5 years of experience, when the driver checks their eligibility to drive electric buses, the system should mark the driver eligible and permit them to operate such buses.
- **Acceptance Criteria 3:** Given a driver holds a Heavy or Public Transport license, when the driver checks their eligibility to drive electric and hybrid buses, then the system should mark the driver eligible and permit them to operate such buses.

User Story 3: As a maintenance technician , I want to update the condition of buses, and add new ones into the network so that it helps the bus network run smoothly.

- **Acceptance Criteria 1:** Given a bus has been approved for the network, the system should add it into the bus repository given it fits the system criterias. Then newly added buses should be retrievable and assignable to bus drivers.
- **Acceptance Criteria 2:** Given a bus' fuel level, capacity or fuel type has changed, the system should allow its retrieval in order to review and update to its new capabilities. Then buses and bus networks would be accordingly assigned depending on those factors.
- **Acceptance Criteria 3:** Given when buses are set to be retired, the system should give the number of all buses within the repository so that the amount of buses to be replaced/required for a smooth network is noted.

User Story 4: As a bus dispatcher, I want to review and update buses to make the most optimal decision in bus scheduling and re-routing depending on circumstances.

- **Acceptance Criteria 1:** Given a bus is needed to replace another, the dispatcher should be able to retrieve the details of buses (capacity, fuel type, fuel level) to decide which bus fits the scope of the network.
- **Acceptance Criteria 2:** Given a bus is temporarily out of order, the bus dispatcher should set capacity to 0 for the time it is non-functional, so that it is non assignable to any routes.
- **Acceptance Criteria 3:** Given a driver is to be assigned a bus for a particular route, retrieve the bus details and review to ensure that it fits the experiences of the driver and route.

User Story 5: As an operations administrator, I want to add a new driver into the system, so that they can be assigned to buses in the transport network.

- **Acceptance Criteria 1:** Given a driver has an ID that is exactly 10 characters long, starts with two digits between 2-9, contains at least two special characters in positions 3-8 and ends with two uppercase letters, when the administrator adds the driver, then the system should store them in the driver repository.
- **Acceptance Criteria 2:** Given a driver has an ID that is duplicate to an existing driver in the repository, when the administrator attempts to add them, then the system should reject the addition and not save them to the file.
- **Acceptance Criteria 3:** Given a driver has an address not following the format Street Number|Street Name|City|State|Country, when the administrator attempts to add them, then the system should reject the addition and throw an error.

User Story 6: As a bus dispatcher, I want to update an existing driver's details, so that the system always has accurate and current information when making scheduling decisions.

- **Acceptance Criteria 1:** Given a driver has more than 10 years of experience, when the dispatcher attempts to change their license type, then the system should reject the update and keep the original license type unchanged.
- **Acceptance Criteria 2:** Given a driver exists in the repository, when the dispatcher updates their address with a valid format, then the change should be saved to the file and retrievable immediately after.
- **Acceptance Criteria 3:** Given the dispatcher attempts to update the driverID or name of the existing driver, when the update is submitted, then the system should reject the change and keep the original ID and name unchanged.

User Story 7: As an operations administrator, I want to assign an eligible driver to an available bus, so that all buses are operated by suitable drivers.

- **Acceptance Criteria 1:** Given that a driver is marked as eligible for handling a bus of a certain capacity limit and a bus is available, when the administrator assigns the driver to the bus, then the system should save the assignment successfully.
- **Acceptance Criteria 2:** Given that a driver is not eligible for the bus' capacity, when the administrator attempts to assign them to the bus, then the system should reject the assignment.
- **Acceptance Criteria 3:** Given that a bus is already assigned to another driver, when the system attempts to assign them to the bus, then the system should reject the assignment.

User Story 8: As a bus driver, I want to report an incident involving a bus, so that the operations team can respond appropriately

- **Acceptance Criteria 1:** Given that a driver enters a valid bus ID, incident type, location, and description, when the report is submitted, then the system should save the incident report.

- **Acceptance Criteria 2:** Given that the incident is marked as serious, when the report is submitted, then the system should flag the bus for a review of the incident level and respond to the incident appropriately.
- **Acceptance Criteria 3:** Given that required incident details are missing, when the driver submits the report, then the system should reject the report and request the missing information.